

FYP Tracking & Accountability System

Defense Mastery Guide — Know Your Project Inside and Out

Student: MANISHIMWE KWIZERA Jean Luc (26972)

Supervisor: ISHIMWE M. Prince

Institution: AUCA — Adventist University of Central Africa, Kigali, Rwanda

How to use this guide. Read Part 1 until you can say the elevator pitch from memory. Read Part 2–6 to understand *what* was built and *why*. Drill Part 8 (anticipated questions) out loud until the answers are automatic. Part 9 is your 30-second cheat answers for when you freeze. Everything here matches the actual code in your repository — if you say it, it is true.

PART 1 — THE ONE-MINUTE PITCH (memorize this)

The **FYP Tracking & Accountability System** is a web platform that records every stage a final-year student passes through — from registration to final defense — capturing **who did what and when**, so that disputes can be resolved from a timestamped record instead of from memory.

The department already keeps good records at the **start** of the journey (registration, case-letter approval) and near the **end** (book submission, defense), but has almost no visibility during **prototype review** and **supervision**. Those blind spots are exactly where students get hurt and where responsibility is today assigned by assumption. My system closes those gaps. Its **first priority** is closing the supervision blind spot; its **second** is joining every stage into one continuous, auditable timeline per student.

It is built as a **React + TypeScript** single-page front end talking over a **REST API** to a **Spring Boot (Java 17)** back end, with **PostgreSQL** as the system of record and **Redis** for caching and token handling. Security is **JWT-based and stateless**, with **role-based access control** for six roles. Every state change is written to a **state-transition log** and an **immutable audit log**, and every email the system sends is stored in a **notification log** with its delivery status — so "I was never told" can be checked against what the system actually sent.

If you remember nothing else: *It is a system of record for the FYP journey whose whole purpose is accountability through evidence.*

PART 2 — THE PROBLEM AND WHY IT MATTERS

2.1 The real-world problem

Final-year project management in the department is **manual and fragmented**: paper forms, spreadsheets, and informal WhatsApp/phone coordination. This causes:

- **The supervision blind spot** — there is no record of supervisor availability, confirmed meetings, attendance, or book sign-off. Nobody can prove supervision actually happened.
- **Untracked attempts** — how many times a student presented a prototype, or resubmitted a rejected proposal, is not reliably recorded with reasons.
- **No proof of communication** — when a student says "nobody told me," there is no evidence either way.
- **No single source of truth** — a student's true stage and history is scattered, so coordination and fair decisions are hard.

2.2 Why it matters

The FYP is the capstone of the degree. When records are weak, students can be **unfairly disadvantaged**, and staff cannot **defend their decisions**. The system turns every event into **timestamped evidence neither side can later deny** — that is the core value.

2.3 The three gaps it closes (say these as a set)

1. **Prototype attempt visibility** — pull schedule, number of presentations, and decisions from the external prototype system (cached so it survives an outage during a dispute).
2. **Proposal rejection trail** — every submission recorded with accept/reject, reason, and attempt number — provable, not anecdotal.
3. **Supervision & panel accountability** — availability, confirmed meetings, sign-off, examiner assignment, and pre-defense/defense outcomes, all timestamped.

PART 3 — ACTORS AND ROLES (6 roles)

The system uses **role-based access control**. Each user has exactly one role, stored on the **users** table and carried in the JWT.

Role	What they do in the system
STUDENT	Submits case letter and proposal; follows meeting dates set by the supervisor; views own state, timeline, supervisor, and notifications.
SUPERVISOR	Posts weekly availability, confirms meeting date/time, records attendance, signs off the book, provides WhatsApp group link(s).
FACILITATOR	The day-to-day coordinator. Views every student's state/timeline, assigns pre-defense/defense examiners, keeps the workflow moving. Operational, not academic decision-making.
HOD (Head of Department)	Uploads the registered-student list, approves case letters, issues requirements, assigns supervisors, reviews proposals, reads prototype data, unlocks proposals.
EXAMINER	An eligible supervisor assigned to a student's pre-defense/defense panel. Records the outcome. Tracked separately from the students they supervise.
SUPERADMIN	System owner. Full access: manages accounts/roles, resets passwords, monitors every log, configures the system, can act anywhere — every action logged against them.

Key subtlety to mention: a person can **supervise some students and examine others at the same time**; the system keeps these assignments and their notifications **distinct** so they never conflict. And a supervisor can **never be the examiner of their own student** (enforced in code).

PART 4 — THE STUDENT JOURNEY (the spine of the system)

Each student occupies **exactly one state at a time**. There are **13 states**. Every transition records the actor and a timestamp, writes an audit entry, and sends an email.

4.1 The states (in order)

REGISTERED → CASE_LETTER_SUBMITTED → CASE_LETTER_APPROVED →
PROTOTYPE_REVIEW → PROTOTYPE_GRANTED → PROPOSAL_UNDER_REVIEW →
PROPOSAL_ACCEPTED → SUPERVISION → BOOK_SUBMITTED → PRE_DEFENSE → DEFENSE →
COMPLETED, plus **WITHDRAWN**.

4.2 Transitions and triggers

From	To	Trigger
REGISTERED	CASE_LETTER_SUBMITTED	Student submits the case-study letter and form
CASE_LETTER_SUBMITTED	CASE_LETTER_APPROVED	HOD approves the letter
CASE_LETTER_APPROVED	PROTOTYPE_REVIEW	HOD issues preliminary requirements
PROTOTYPE_REVIEW	PROTOTYPE_REVIEW	Needs refinement — student re-presents (attempt counted in proto_attempts)
PROTOTYPE_REVIEW	PROTOTYPE_GRANTED	Prototype granted
PROTOTYPE_GRANTED	PROPOSAL_UNDER_REVIEW	Student submits the proposal book
PROPOSAL_UNDER_REVIEW	PROPOSAL_UNDER_REVIEW	Book rejected — student resubmits (attempt + reason recorded)
PROPOSAL_UNDER_REVIEW	PROPOSAL_ACCEPTED	Proposal accepted
PROPOSAL_ACCEPTED	SUPERVISION	Department assigns a supervisor
SUPERVISION	BOOK_SUBMITTED	Supervisor signs off; book submitted
BOOK_SUBMITTED	PRE_DEFENSE	Examiner(s) assigned; scheduled
PRE_DEFENSE	DEFENSE	Examiner records pre-defense as cleared
DEFENSE	COMPLETED	Defense panel records the result (passed)
<i>Any non-terminal state</i>	WITHDRAWN	Student withdrawn

4.3 The two important rules

- **WITHDRAWN** is allowed from **any non-terminal** state and is **terminal** (no transitions out).
- **COMPLETED** is **terminal**.
- **FLAGGED** is **not a state** — it is a boolean on the student record, so any authorised actor can flag a student while they stay in their current state.

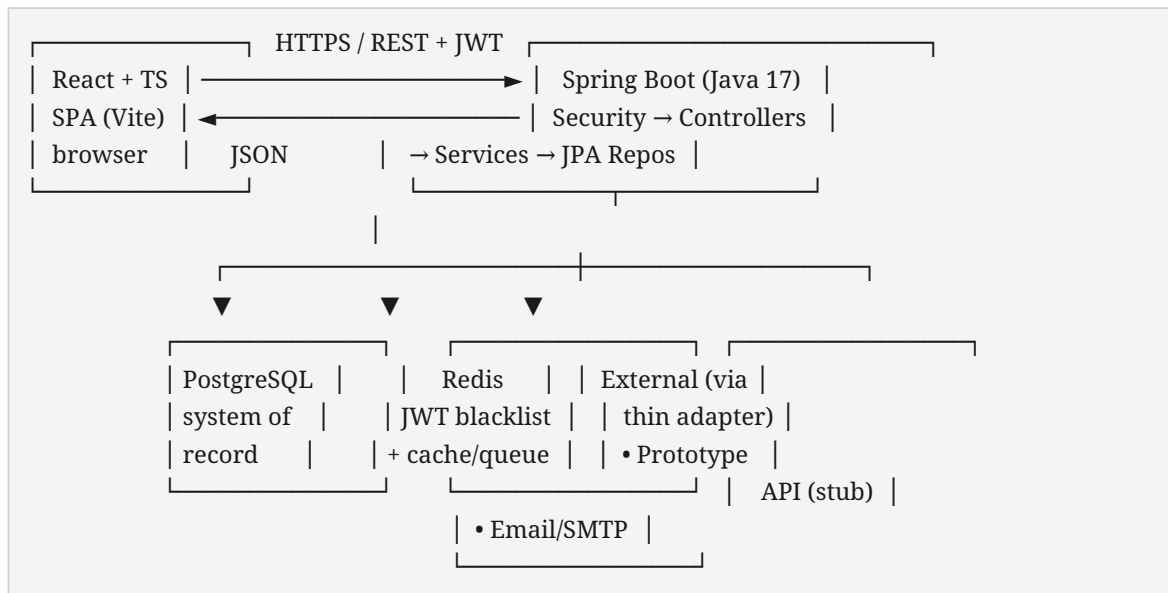
4.4 How it is enforced (this is the part to be proud of)

The state machine lives in `StudentStateService.java` as an explicit map of **allowed transitions**:

```
Map<StudentState, Set<StudentState>> ALLOWED = Map.ofEntries(
    Map.entry(REGISTERED,      EnumSet.of(CASE_LETTER_SUBMITTED)),
    Map.entry(PROTOTYPE_REVIEW, EnumSet.of(PROTOTYPE_REVIEW, PROTOTYPE_GRANTED)),
    Map.entry(PROPOSAL_UNDER_REVIEW, EnumSet.of(PROPOSAL_UNDER_REVIEW,
    PROPOSAL_ACCEPTED)),
    ... );
```

Any transition not in this map throws `InvalidStateTransitionException`. In **one transactional method** the service: validates the move → increments `proto_attempts` on a prototype self-loop → saves the new state and `state_entered_at` → writes a `StateTransition` row → writes an `AuditLog` row → sends the milestone email. So the journey **cannot be corrupted** by an out-of-order action, and every legal move leaves a trail.

PART 5 — ARCHITECTURE (draw this on the whiteboard if asked)



5.1 Layered back-end (classic Spring layering)

- **Controller layer** — REST endpoints, validates input, enforces role access with `@PreAuthorize`. Returns **DTOs**, never entities.
- **Service layer** — business logic and transactions (`@Transactional`): the state machine, proposal rules, panel rules, supervision, notifications, audit.
- **Repository layer** — Spring Data JPA interfaces over the entities.
- **Entity layer** — JPA-mapped tables.
- **Security layer** — JWT filter + utility, runs before the controllers.

5.2 Why this architecture (say "separation of concerns")

Each layer has one job, so the system is **testable, maintainable, and the API contract is decoupled from the database**. DTOs stop Hibernate lazy-loading and internal fields from leaking to the client.

5.3 The two external integrations sit behind adapters

The **prototype-review API** and the **email provider** are each behind a thin adapter, so if the other side changes, only **one component** changes. The prototype API contract is not finalised yet, so it runs against a **stub** (`PROTO_API_ENABLED=false`) and swaps in later with no change elsewhere. This is a deliberate, defensible design — not an unfinished piece.

PART 6 — TECHNOLOGY STACK AND WHY EACH CHOICE

Defense panels reward **reasoned** choices. For every technology, know the one-line "why."

Layer	Technology	Why this choice
Front end	React 19 + TypeScript + Vite	Component reuse for role dashboards; TypeScript's static types catch bugs before runtime; Vite gives fast dev builds; SPA gives a responsive, app-like UX.
Back end	Spring Boot 3.2.5, Java 17	Mature, enterprise-grade, secure-by-default ecosystem; first-class Security, Data JPA, Validation, Mail, Actuator. Ideal for a transactional system of record.
Persistence	PostgreSQL 16	Relational data with foreign-key integrity and ACID transactions — essential when the data is the evidence. Comfortably handles 500–1000 students/cycle.
ORM	Spring Data JPA + Hibernate	Maps objects to tables, removes boilerplate SQL, gives transaction management.
Migrations	Flyway	Versioned, repeatable schema (V1, V2, V3). The DB schema has a single source of truth; runs automatically on startup.
Cache / tokens	Redis 7	In-memory store used for the JWT logout blacklist (with TTL auto-expiry), and available for caching timelines and a notification queue.
Auth	Spring Security + JWT (jjwt 0.11.5)	Stateless authentication that scales and fits a SPA + REST API. BCrypt (cost 12) for password hashing.
Email	Spring Mail (JavaMailSender)	Sends milestone notifications over SMTP; sent asynchronously so it never blocks a request.
Excel import	Apache POI 5.2.5	Reads the HOD's .xlsx student list to bulk-create accounts.
Build	Maven	Standard Java dependency

		management and build lifecycle.
--	--	---------------------------------

Java 17 specifically: it is a Long-Term-Support release, required by Spring Boot 3.x (which is built on Jakarta EE 9+/Spring 6).

PART 7 — THE FOUR HARD PARTS (deep dives the panel will probe)

7.1 Security — authentication and authorization

How login works:

4. `POST /api/auth/login` with email + password (the **only** public endpoint besides the health check).
5. Spring Security's `AuthenticationManager` checks the password against the `BCrypt` hash.
6. On success, `JwtUtil` issues a signed JWT containing: `subject = email`, claims `role` and `userId`, a unique `jti`` (token id), issued-at, and a 24-hour expiry, signed with `HMAC-SHA256`.
7. The client stores the token and sends it as `Authorization: Bearer <token>` on every request.

How each request is authorized:

- `JwtAuthFilter` (a `OncePerRequestFilter`) runs before the controllers. It reads the Bearer token, validates the signature/expiry, checks the **Redis blacklist** by `jti`, loads the user, and puts an `Authentication` into the `SecurityContext`.
- Sessions are **STATELESS** (`SessionCreationPolicy.STATELESS`) — no server-side session; the token is the session.
- **CSRF is disabled** — correct for a stateless token API (CSRF protection matters for cookie-based sessions, which we do not use).
- Method-level `@PreAuthorize("hasAnyRole('HOD','FACILITATOR',...)" )` on controllers enforces who can call what. `User` implements `UserDetails` and exposes its authority as `ROLE_<ROLE>`.

Logout / token revocation (the clever bit): JWTs are normally hard to revoke because they're stateless. On logout we put the token's `jti` into **Redis with a TTL equal to the token's remaining lifetime** — so it is rejected until it would have expired anyway, then Redis auto-cleans it. Best of both worlds: stateless performance, real revocation.

One design note worth volunteering: `JwtAuthFilter` injects `UserRepository` directly instead of `UserDetailsService` to avoid a circular dependency with `SecurityConfig`.

7.2 The proposal 3-attempt rule

In `ProposalService` (`MAX_REJECTIONS = 3`):

- A proposal can only be submitted from `PROTOTYPE_GRANTED` or `PROPOSAL_UNDER_REVIEW`, and only if not **locked**, and only if there is no existing **PENDING** attempt.
- On **reject**, a **reason is mandatory**; the attempt is stored with its number and reason.
- When **rejections reach 3**, `proposal_locked` is set **true** — the student **cannot resubmit** until the **HOD unlocks** it (`POST /api/proposals/{id}/unlock`), which is itself audited and emailed.
- This is exactly the "proposal rejection trail" gap — now provable.

7.3 Panel rules (pre-defense / defense)

In `PanelService`:

- The assigned user **must have the EXAMINER role**.

- An examiner **cannot be the student's own supervisor** (explicit check).
- **No duplicate** assignment (same student + examiner + panel type).
- **Outcome validation by panel type:** pre-defense outcome must be **CLEARED**; defense outcome must be **PASSED / REFERRED / FAILED** (never **CLEARED**).
- **Auto state transition:** recording **CLEARED** on a pre-defense moves PRE_DEFENSE → DEFENSE; recording **PASSED** on a defense moves DEFENSE → COMPLETED.
- An examiner **can only record outcomes on their own** assignments; an assignment **cannot be deleted once an outcome is recorded** (preserves the record).

7.4 Notifications and accountability logs

- `EmailService.send(...)` is `@Async` (`@EnableAsync` in `AsyncConfig`), so email I/O never blocks the request or its transaction.
- Every email writes a `NotificationLog` row: it starts **PENDING**, then becomes **SENT** (with `sentAt`) or **FAILED** (with the error message and an incremented `retryCount`).
- The **audit log** (`AuditService`) records who did what, on which entity, with detail and timestamp — an **append-only** record. Between the **state transitions**, the **notification log**, and the **audit log**, the system has three independent evidence trails. That redundancy *is* the accountability design.

PART 8 — ANTICIPATED PANEL QUESTIONS (drill these out loud)

Q: In one sentence, what is your project?

A system of record for the final-year-project journey that timestamps every state change, supervision event, and notification so disputes are resolved from evidence rather than memory.

Q: What problem does it actually solve?

It closes the supervision and prototype-review blind spots and gives every student one continuous, auditable timeline, replacing scattered paper/WhatsApp coordination.

Q: Why Spring Boot and not Node/PHP/Django?

Because the data is the evidence, I needed strong transactions, mature security, and reliability. Spring Boot gives enterprise-grade Security, Data JPA, validation and transaction management out of the box, on a statically-typed, LTS Java platform.

Q: Why PostgreSQL and not MongoDB?

The domain is **highly relational** — students, supervisors, transitions, attempts, panels all reference each other — and I need **foreign-key integrity and ACID guarantees**. A document store would weaken exactly the integrity an accountability system depends on.

Q: Why JWT instead of server sessions?

Stateless auth scales horizontally and fits a SPA + REST API: no shared session store. The classic downside — revocation — I solved with a **Redis blacklist keyed by the token's jti**, with a TTL equal to the token's remaining life.

Q: If JWT is stateless, how do you log a user out?

I blacklist the token's **jti** in Redis until its natural expiry. The auth filter rejects any blacklisted token. Redis evicts it automatically when the TTL ends.

Q: How do you stop a student jumping straight to DEFENSE?

The state machine only permits transitions defined in an explicit allowed-transitions map. Anything else throws **InvalidStateTransitionException**. The journey integrity is enforced in one place and is unit-testable.

Q: How do you guarantee the audit trail can't be quietly bypassed?

Every state change goes through the single **transition(...)** method, which writes the state-transition row **and** the audit row **and** the notification inside the same transaction. There is no path that changes state without leaving all three traces.

Q: How is a password stored?

BCrypt with strength factor 12 — an adaptive hash with a built-in per-password salt. We never store or log plaintext.

Q: How do you keep an examiner from grading their own supervisee?

`PanelService.assign` rejects assigning a student's own supervisor as examiner, and examiners can only record outcomes on panels they're assigned to. Supervision and examination are tracked as separate relationships.

Q: What happens if the email server is down?

Email is asynchronous, so the request still succeeds. The notification is logged as **FAILED** with the error and a retry count, so we keep proof of the attempt and can retry — nothing is silently lost.

Q: How does the HOD get students into the system?

The HOD uploads an Excel file; Apache POI parses it and creates a `User` + linked `Student` for each row, with the default password set to the registration number and the state initialised to REGISTERED.

Q: Why DTOs instead of returning entities?

To decouple the API from the database, avoid leaking internal fields and lazy-loaded associations, and control exactly what the client sees.

Q: Why Flyway with `ddl-auto=validate`?

Flyway migrations are the single source of truth for the schema; `validate` makes Hibernate confirm the entities match the migrated schema without ever altering the database itself — safe and reproducible.

Q: How does this respect data protection law?

It holds only registrar-supplied personal data (names, IDs, emails), restricts access by role, keeps personal data for one year then removes/anonymises it, and logs every read/change — aligned with Rwanda's Law N° 058/2021 on personal data protection.

Q: How does it scale to 1000 students?

PostgreSQL with proper indexing handles this volume comfortably; Redis caches frequently-read data like timelines; email is queued/async so it never blocks. The design target is 500–1000 students and 10–50 supervisors per cycle.

Q: What was the hardest part?

Designing the state machine so the journey could never be corrupted while still recording the representation loops (prototype) and rejection loops (proposal) that the accountability story depends on — and doing all of it inside one transaction with the audit and notification writes.

PART 9 — 30-SECOND CHEAT ANSWERS (when you freeze)

- **What is it?** "A system of record for the FYP journey; its purpose is accountability through timestamped evidence."
- **Stack?** "React + TypeScript front end, Spring Boot (Java 17) REST back end, PostgreSQL, Redis, JWT auth."
- **Core feature?** "A 13-state student journey enforced by a state machine, with every transition logged."
- **Security?** "Stateless JWT, role-based access with @PreAuthorize, BCrypt-12 passwords, Redis blacklist for logout."
- **Accountability?** "Three trails — state transitions, an immutable audit log, and a notification log with delivery status."
- **External systems?** "Prototype-review API and email, each behind a thin adapter; prototype runs on a stub until its contract is final."
- **Why relational DB?** "The data is relational and must have integrity and ACID guarantees — it *is* the evidence."

PART 10 — LIMITATIONS & FUTURE WORK (own these confidently)

Presenting scoped limitations as **deliberate decisions** signals maturity:

- **Prototype API is a stub** — the external contract isn't finalised, so it runs behind an adapter against an assumed contract; only the adapter changes when the real API arrives. (A design choice, not a gap.)
- **WhatsApp uses free group links**, not a paid messaging API — the system stores and surfaces the link(s); it does not administer the groups.
- **Email is the system's own notification channel; SMS and an in-app notification centre are "Could"/future** items in the MoSCoW prioritisation.
- **Analytics on bottlenecks** (e.g., average time per phase) is future insight, not core.
- **One-year retention** of personal data is enforced by design; further data-protection automation can follow.
- **Future hardening**: broader automated test coverage, fuller Redis caching of timelines, and a delivery/retry worker for failed emails.

PART 11 — GLOSSARY (say these correctly)

- **JWT (JSON Web Token):** a signed, self-contained token carrying the user's identity and role; verified on every request.
- **jti:** the unique ID claim inside a JWT; used as the Redis blacklist key for logout.
- **BCrypt:** an adaptive password-hashing function with a built-in salt; "cost 12" sets how slow (and thus hard to brute-force) it is.
- **Stateless authentication:** the server keeps no session; the token proves who you are each time.
- **RBAC (Role-Based Access Control):** permissions granted by role, enforced with [@PreAuthorize](#).
- **State machine:** a model where an entity is in one state at a time and may only move along defined transitions.
- **DTO (Data Transfer Object):** a purpose-built object for the API request/response, separate from the database entity.
- **ORM / JPA / Hibernate:** maps Java objects to relational tables; JPA is the standard, Hibernate the implementation.
- **Flyway migration:** a versioned SQL script that evolves the database schema reproducibly.
- **ACID:** Atomicity, Consistency, Isolation, Durability — the transaction guarantees a relational DB provides.
- **Idempotent / append-only log:** the audit and notification logs are only ever added to, never edited — that is what makes them trustworthy evidence.
- **Adapter:** a thin layer isolating an external system so a change there affects only one component.

End of guide. If you can deliver Part 1 from memory, justify every row of the Part 6 table, and answer Part 8 without hesitating, you will defend this project with confidence.